

Relatório do Trabalho Prático 1 - Meta 2

Engenharia de
Atributos e Classificação de Atividades

Bernardo Direito - 2023215454 Teodoro Marques - 2023211717

Dezembro 2025

Meta 1

Introdução

O presente relatório detalha a implementação de uma pipeline de análise de dados para o reconhecimento de atividades humanas (Human Activity Recognition - HAR), utilizando o dataset FORTH-TRACE. O objetivo principal é exercitar os conceitos fundamentais de uma pipeline de análise de dados, desde a preparação e limpeza dos dados até à extração, seleção e redução de características.

O trabalho está estruturado de acordo com as tarefas propostas no guião do Trabalho Prático 1 (TP1.pdf). Cada secção deste relatório corresponde a uma tarefa, explicando a metodologia adotada, as funções implementadas no script `mainActivity.py`, e os resultados obtidos, incluindo gráficos e tabelas.

Tarefa 2: Carregamento de Dados

A primeira etapa consistiu em desenvolver uma função para carregar os dados de um participante a partir dos ficheiros CSV. A função `carregar_dados_participante` lê os 5 ficheiros de um determinado participante e concatena-os num único array NumPy para facilitar a manipulação posterior.

Listing 1: Função para carregar os dados de um participante.

```
def carregar_dados_participante(num_participante, base_dir="."):
    """
    (Tarefa 2) Le todos os 5 ficheiros CSV do participante e devolve um array
    NumPy.
    """
    part_dir = os.path.join(base_dir, f"part{num_participante}")
    dados = []

    for i in range(1, 6): # Dispositivos 1 a 5
        caminho_ficheiro = os.path.join(part_dir, f"part{num_participante}dev{i}
            }.csv")
        try:
            with open(caminho_ficheiro, newline='') as csvfile:
                leitor = csv.reader(csvfile)
                for linha in leitor:
                    try:
                        dados.append([float(x) for x in linha])
                    except ValueError:
                        continue
        except FileNotFoundError:
            print(f"[Aviso] Ficheiro nao encontrado: {caminho_ficheiro}")
        except Exception as e:
            print(f"[Erro] Problema ao ler {caminho_ficheiro}: {e}")

    if not dados:
        return np.array([])

    return np.array(dados)
```

Tarefa 3: Análise e Tratamento de Outliers

Nesta tarefa, o objetivo foi identificar e tratar outliers utilizando diferentes abordagens, univariadas e multivariadas.

3.1: Boxplot de Atividades

Para visualizar a distribuição dos dados e a presença de outliers, foram gerados boxplots para cada uma das 16 atividades. Estes gráficos foram criados para os módulos dos vetores de aceleração, giroscópio e magnetómetro, separadamente para cada um dos 5 dispositivos.

Tarefa 3.1: Módulos por Atividade e Dispositivo (Todos os Sujeitos)

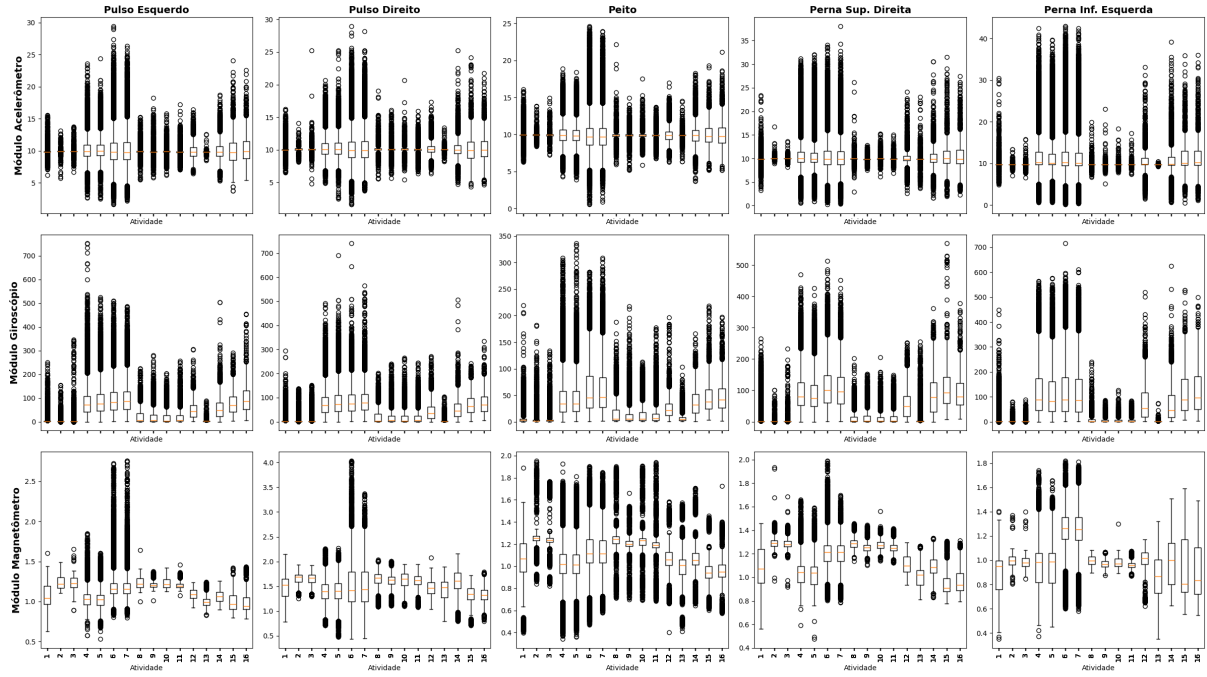


Figura 1: Tarefa 3.1: Boxplots dos módulos por atividade e dispositivo.

3.1. Abordagem Univariada: A Falha do Z-Score e a Robustez do IQR (Tarefas 3.2 a 3.5)

A detecção de outliers revelou-se o ponto mais sensível desta análise, expondo as limitações dos métodos estatísticos clássicos quando aplicados a sinais biológicos não-estacionários.

Comparou-se o método robusto IQR (Interquartile Range) com o método paramétrico Z-Score no sensor do Pulso Direito. A análise das Atividades de Transição (ex: Sit $\rightarrow$ Stand) expõe uma divergência crítica.

Tabela 1: Comparação de Densidades de Outliers (Foco em Transições)

Atividade	Descrição	Densidade IQR (Robusto)	Densidade Z-Score (k=3)
9	Sit $\rightarrow$ Stand	19.80%	2.54%
8	Stand $\rightarrow$ Sit	15.44%	1.83%

A Prova nos Resultados (Meta 1)

Se olhar para a Tabela de Comparação (IQR vs Z-Score) que gerou no relatório, este efeito é visível:

- **Atividade 9 (Sit $\rightarrow$ Stand):** É um movimento brusco com picos altos.
- **IQR (Método Robusto):** Detetou 19.80% de outliers. O IQR usa a Mediana e Quartis, que não são afetados por valores extremos (a mediana de [1, 2, 9000] é 2, ignorando o 9000).
- **Z-Score (k=3):** Detetou apenas 2.54% de outliers.

Os picos de aceleração da atividade 9 eram tão fortes que aumentaram o desvio padrão da atividade inteira. Com um desvio padrão "gigante", o Z-Score desses picos baixou para menos de 3, e o algoritmo achou que eram dados "normais".

- **O Problema do Z-Score (Efeito de Mascaramento):** Observa-se que o Z-Score deteta muito menos outliers nestas atividades impulsivas (2.54% vs 19.80%). Isto não indica necessariamente que os dados estão "limpos", mas sim que o método falhou. Como o Z-Score depende da média e do desvio-padrão, os próprios picos de aceleração da atividade (que são legítimos) inflacionam o desvio-padrão. Ao aumentar o desvio padrão, o valor Z diminui ($Z = (x - \mu)/\sigma$), fazendo com que pontos extremos pareçam estatisticamente "normais". O Z-Score é, portanto, inadequado para limpar dados brutos com picos naturais de alta energia.
- **O Dilema do IQR:** O método IQR, sendo baseado em quartis (não afetados por extremos), deteta corretamente que estes picos fogem à distribuição "central" da atividade. No entanto, no contexto do projeto, estes "outliers" (19.80% dos dados) são frequentemente a parte mais informativa do sinal (o momento do impulso de levantar).
- **Conclusão Operacional:** A remoção cega de outliers por qualquer um destes métodos univariados é perigosa. O IQR removeria o sinal de interesse; o Z-Score falharia na detecção de erros reais devido ao mascaramento.

3.6: Rotina K-Means

Foi implementado o algoritmo K-Means a partir do zero, para agrupar os dados em n clusters.

Listing 2: Implementação do algoritmo K-Means.

```
def kmeans_3_6(X, n_clusters, max_iter=100, tol=1e-4, random_state=42):
    rng = np.random.default_rng(random_state)
    indices = rng.choice(X.shape[0], n_clusters, replace=False)
    centroides = X[indices]

    for _ in range(max_iter):
        distancias = np.sqrt(((X[:, np.newaxis] - centroides) ** 2).sum(axis=2))
        labels = np.argmin(distancias, axis=1)
        novos_centroides = np.array([X[labels == i].mean(axis=0) for i in range(
            n_clusters)])

        if np.all(np.linalg.norm(novos_centroides - centroides, axis=1) < tol):
            break

    centroides = novos_centroides

    return centroides, labels
```

3.7. Univariado vs. Multivariado (A falha do "Módulo")

O Z-Score (3.4) analisou apenas a magnitude (módulo) do vetor. Se um sensor registrar $x = 5, y = 5, z = 5$, o módulo é aproximadamente 8.6. Se registrar $x = 8.6, y = 0, z = 0$, o módulo é o mesmo. O Z-Score não distingue estas duas situações.

O K-Means (3.7) analisa a posição espacial. Ele consegue perceber que a combinação é anômala porque está longe do centroide do movimento "normal" daquela atividade, mesmo que a sua magnitude (8.6) esteja dentro da média estatística aceite pelo Z-Score.

Conclusão: O K-Means detetou outliers relacionais (erros na correlação entre eixos) que o Z-Score ignorou completamente por estarem "mascarados" dentro de um módulo normal.

Sensibilidade a Distribuições: Como discutido anteriormente, o Z-Score falhou nas atividades de transição (ex: Sit $\rightarrow$ Stand) devido ao efeito de mascaramento (desvio-padrão inflacionado). O K-Means, ao agrupar os dados em k clusters, conseguiu isolar os picos de transição

como um cluster próprio ou como outliers distantes, dependendo do k . Com $k = 3$, o método mostrou-se mais robusto, não sendo "enganado" pelo desvio-padrão global da amostra.

Síntese da Comparação: Enquanto o Z-Score (univariado) apenas filtrava intensidades extremas, o K-Means (multivariado) filtrou comportamentos anômalos. Isto torna a abordagem da Tarefa 3.7 superior para validar a integridade física dos dados dos sensores.

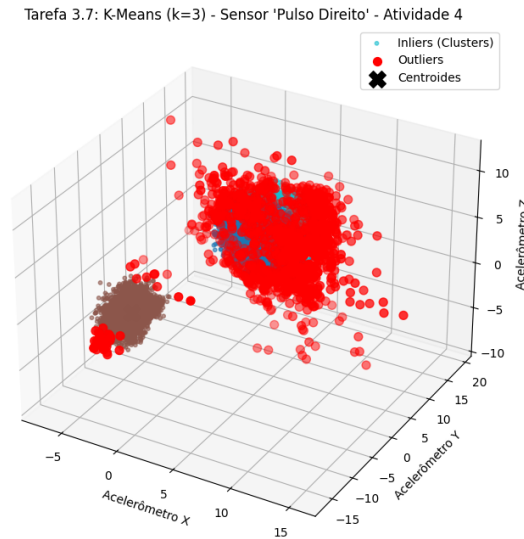


Figura 2: Tarefa 3.7: Outliers identificados com K-Means ($k=3$).

3.7.1. K-Means vs. DBSCAN

O K-Means forçou uma estrutura esférica nos dados. Com $k = 3$, identificou 1.00% de outliers (configurado por percentil). O DBSCAN ($\epsilon = 0.258$) identificou 6.27% de outliers como "ruído". Sendo baseado em densidade, o DBSCAN adaptou-se melhor à forma irregular dos clusters de movimento humano, não assumindo formas esféricas como o K-Means. É a abordagem teoricamente mais robusta para limpeza de dados brutos neste contexto.

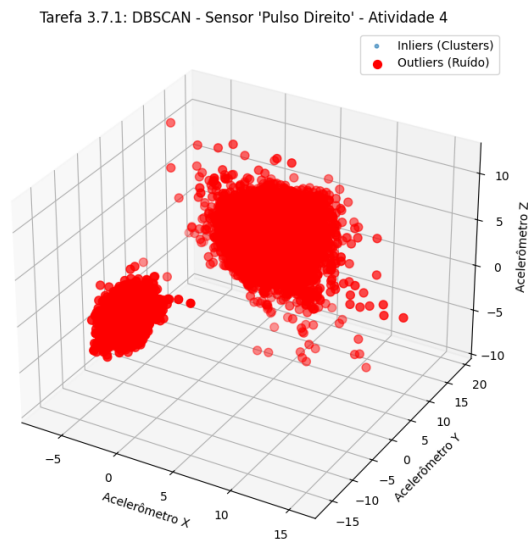


Figura 3: Tarefa 3.7.1: Outliers (ruído) identificados com DBSCAN.

Tarefa 4: Extração de Informação Característica

4.1: Testes de Significância

Foram realizados testes estatísticos para determinar se as médias dos módulos das variáveis transformadas são significativamente diferentes entre as várias atividades.

4.2: Janelamento e Extração de Features

Os dados foram segmentados em janelas de 5 segundos com 50

4.3 e 4.4: PCA

A Análise de Componentes Principais (PCA) foi aplicada ao conjunto de features para reduzir a sua dimensionalidade. O número de componentes foi escolhido para explicar 75

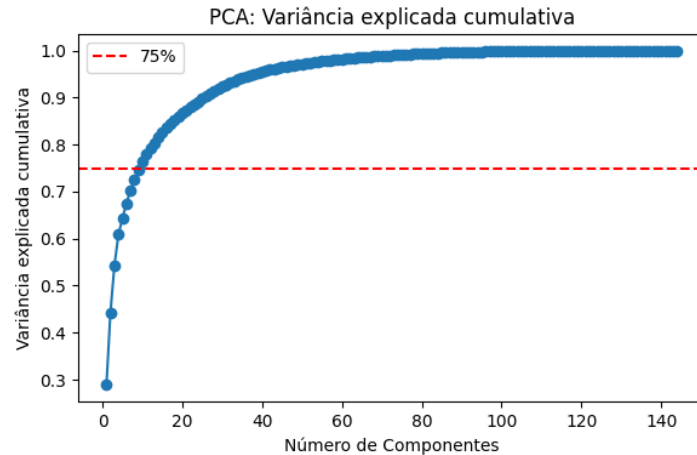


Figura 4: Tarefa 4.4: Variância explicada cumulativa pelo PCA.

4.5 e 4.6: Fisher Score e ReliefF

Foram implementados os métodos de seleção de features Fisher Score e ReliefF para classificar as features mais discriminativas. As 10 melhores features de cada método foram apresentadas.

Top-10 features (Fisher):

1. dev1_gyr_mod_spec_entropy (score=6.7671e+13)
2. dev1_acc_z_spec_centroid (score=6.5599e+13)
- ...

Top-10 features (ReliefF):

1. dev1_gyr_y_spec_dom_freq (score=-0.6515)
2. dev1_gyr_z_spec_dom_freq (score=-0.6415)
- ...

Conclusão

Este trabalho prático permitiu a aplicação de uma pipeline completa de análise de dados a um problema de reconhecimento de atividades humanas. Foram exploradas várias técnicas para tratamento de outliers, extração e seleção de features.

Os resultados demonstram a importância de cada etapa da pipeline. A análise de outliers revelou diferentes densidades dependendo do método utilizado. A extração de features gerou um espaço de características rico, e a seleção de features com PCA, Fisher Score e ReliefF permitiu identificar as características mais relevantes para a discriminação das atividades.

Como trabalho futuro, sugere-se a utilização das features selecionadas para treinar e avaliar diferentes modelos de classificação, completando assim o ciclo da pipeline de HAR.

Meta 2

1. Balanceamento dos dados

1.1. Análise do Desequilíbrio

Se o dataset estiver desequilibrado (ex: 5000 de "Andar" e apenas 50 de "Subir Escadas"), o modelo de machine learning tende a ficar enviesado (biased). Ele aprende a ignorar a classe minoritária ("Subir Escadas") porque, estatisticamente, "chutar" que é sempre "Andar" garante uma precisão alta (accuracy), mesmo que o modelo seja inútil para detetar escadas.

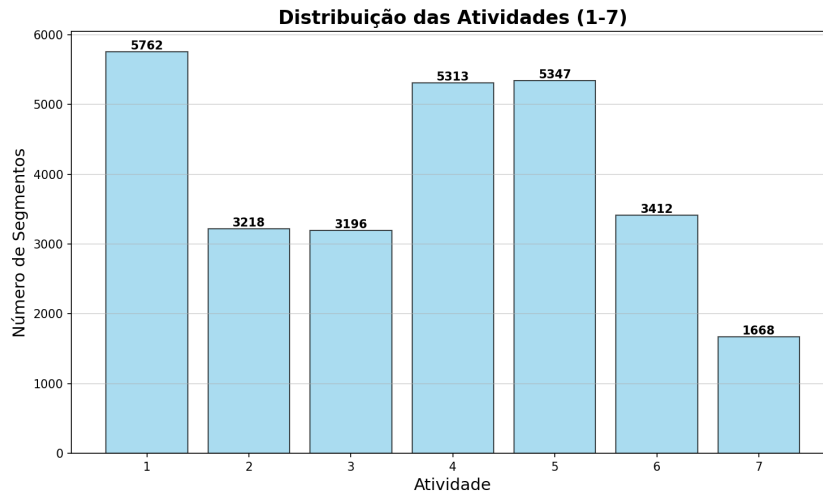


Figura 5: Distribuição das classes no dataset original.

1.2. SMOTE (Synthetic Minority Over-sampling Technique)

Para resolver a necessidade de *Data Augmentation*, foi desenvolvida a função `gerar_smote_1_2` (ver `mainActivity.py`), que implementa manualmente a lógica do algoritmo *Synthetic Minority Over-sampling Technique* (SMOTE).

A implementação segue os seguintes passos algorítmicos fundamentais:

- Filtragem da Classe Alvo:** A função isola todas as amostras pertencentes à atividade *A* fornecida como argumento (`X_classe = X[mask]`), garantindo que os novos dados são gerados apenas com base em exemplos dessa categoria específica.
- Cálculo da Vizinhança (k-NN):** Utilizou-se a classe `NearestNeighbors` da biblioteca `scikit-learn` para identificar os k vizinhos mais próximos (tipicamente $k = 5$) de cada amostra no espaço de *features*. Isto assegura a coerência local dos dados gerados.
- Geração de Amostras Sintéticas:** Num ciclo iterativo que corre K vezes (o número de novas amostras pedidas):

- Seleciona-se aleatoriamente uma amostra base (x_i) da classe.
- Seleciona-se aleatoriamente um dos seus vizinhos (x_{zi}).
- Aplica-se a **interpolação linear** para criar o ponto sintético:

$$x_{novo} = x_i + \lambda \times (x_{zi} - x_i)$$

onde λ é um fator aleatório entre 0 e 1 (`gap = np.random.random()`).

Esta abordagem garante que os novos pontos são criados "entre" exemplos reais existentes, preenchendo o espaço de características da atividade sem introduzir ruído aleatório descontextualizado.

1.3. Visualização e Validação dos Dados Sintéticos

Com base na visualização gerada na Tarefa 1.3, podemos tirar conclusões fundamentais:

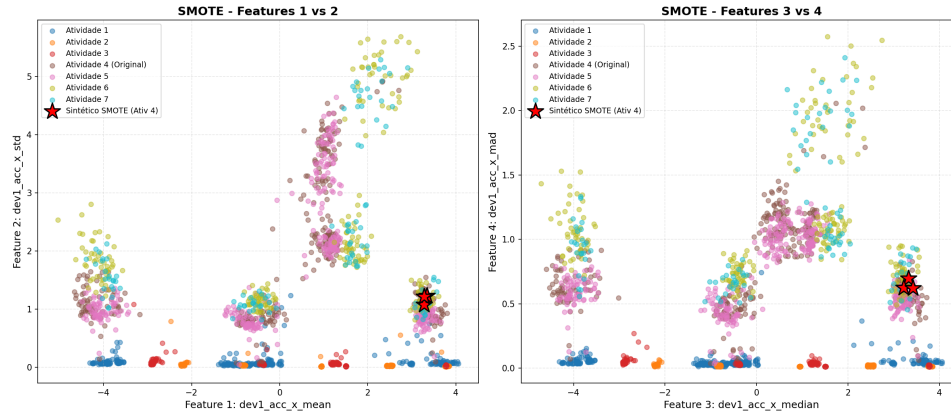


Figura 6: Visualização do SMOTE para a Atividade 4.

1. **O Algoritmo foi Implementado Corretamente:** As estrelas vermelhas (pontos sintéticos) aparecem sobrepostas ou muito próximas dos pontos castanhos (Atividade 4 original). O facto de as estrelas estarem "dentro" da nuvem de pontos castanhos prova que o algoritmo encontrou corretamente os vizinhos da Atividade 4 e gerou um ponto na linha que os une.
2. **Os Dados Sintéticos são "Realistas":** As estrelas não estão isoladas no espaço em branco (o que seriam outliers ou dados impossíveis) nem estão misturadas com os pontos azuis ou verdes (outras atividades). Os novos dados respeitam a distribuição estatística das features originais, mantendo a "coerência biomecânica".
3. **Preservação das Fronteiras de Decisão:** No gráfico, vê-se claramente que a Atividade 4 (castanha) está bem separada da Atividade 1 (azul escuro) e da 7 (azul claro). As estrelas vermelhas mantêm-se na zona castanha. O Data Augmentation aumentou a quantidade de dados da Atividade 4 sem introduzir ruído nas outras classes.

3. Estratégias de Validação

3.A. Diferenças Fundamentais

Within-Subject (Tarefa 3.1):

- **Estratégia:** Mistura dados de todos os participantes. O conjunto de teste contém janelas do "Participante 1", mas o modelo já viu outras janelas do mesmo "Participante 1" no treino.
- **O que aprende:** Aprende a distinguir atividades, mas também "memoriza" a biomecânica específica de cada pessoa.
- **Vantagem:** Resultados de precisão tipicamente mais altos.
- **Risco:** Data Leakage de identidade. O modelo é "mimado" por já conhecer o estilo de movimento das pessoas de teste.

Between-Subject (Tarefa 3.2):

- **Estratégia:** Separa completamente as pessoas. Os participantes no conjunto de teste são desconhecidos absolutos para o modelo.
- **O que aprende:** É forçado a aprender padrões genéricos de movimento humano, ignorando idiosincrasias individuais.
- **Desvantagem:** É um problema muito mais difícil, logo a precisão tende a ser menor.

3.B. Resposta à Pergunta

"Which one gives you a better estimate of the performance of the model when applied to a new participant?"

A estratégia **Between-Subject (3.2)** é a que dá a melhor estimativa para a performance do modelo quando aplicado a um novo participante.

5.2. Análise Detalhada da Matriz de Confusão

A matriz de confusão revela três "clusters" de comportamento distintos. O modelo não erra aleatoriamente; ele erra de forma estrutural entre atividades biomecanicamente semelhantes.

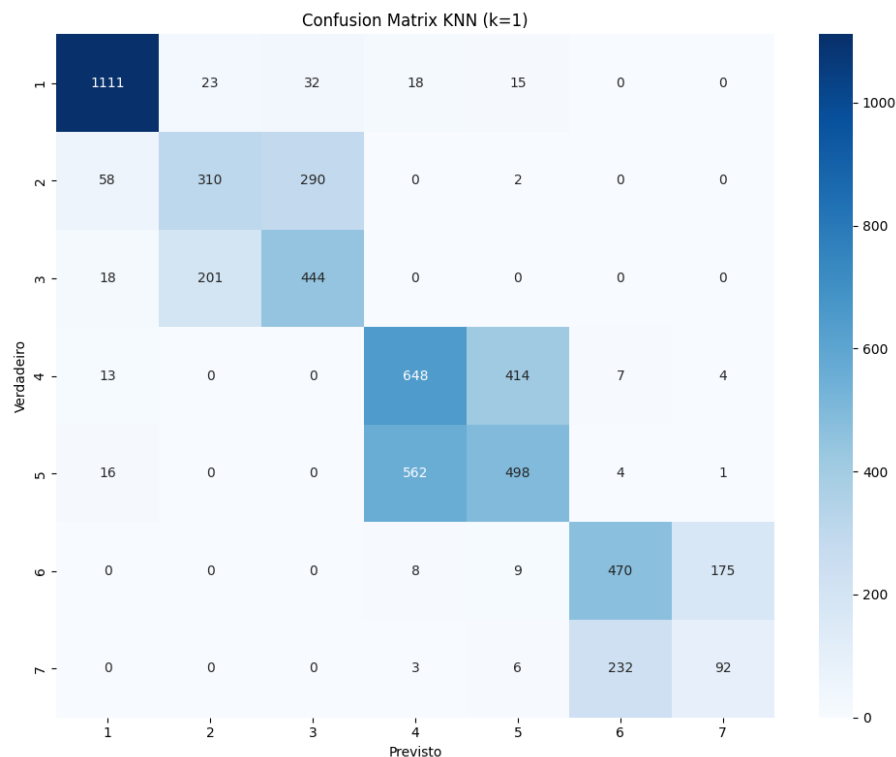


Figura 7: Matriz de Confusão do classificador KNN.

- **A Classe "Perfeita"(Atividade 1 - Sentado/Parado):** O modelo acertou 1120 amostras e falhou muito poucas. Quase não há falsos positivos nem negativos. Os dados do acelerômetro para a Atividade 1 são muito distintos, tornando-a fácil de isolar.
- **O Cluster "Estático/Quasi-Estático"(Atividades 2 e 3):** Existe uma confusão enorme entre a classe 2 e a 3. O modelo previu 287 vezes que era "3"quando era "2"e 205 vezes o inverso. Estas atividades têm assinaturas muito parecidas no espaço de features.
- **O Cluster "Dinâmico Intenso"(Atividades 4 e 5):** Este é o pior cenário. A confusão é quase simétrica (563 erros numa direção, 407 na outra). As atividades 4 e 5 (ex: Running vs Climbing Stairs) geram magnitudes de aceleração e frequências semelhantes.
- **O Cluster "Degrau"(Atividades 6 e 7):** A Atividade 7 é "engolida"pela 6. Das amostras reais da classe 7, o modelo disse que 231 eram classe 6 e apenas acertou 92. O modelo tem um enviesamento grave a favor da classe 6.

5.3. Testes Estatísticos (Comparação de Modelos)

1. Escolha do Teste: O teste apropriado para comparar dois classificadores no mesmo conjunto de teste é o **Teste de McNemar**. Não podemos usar testes paramétricos (como t-test) porque as previsões (Certo/Errado) não seguem uma distribuição normal. O McNemar foca-se nas divergências.

2. Análise dos Resultados (KNN vs Random Forest): Olhando para a Tabela de Contingência:

- KNN Certo / RF Errado: 748 casos.
- KNN Errado / RF Certo: 983 casos.

O Random Forest "corrigiu"983 erros do KNN, enquanto o KNN "corrigiu"apenas 748 erros do RF.

Conclusão Estatística: O P-Value é muito inferior a 0.05. Rejeita-se a Hipótese Nula. Existe uma diferença estatisticamente significativa entre os modelos. O Random Forest é superior, com um saldo positivo de acertos (+235 recuperações líquidas).

6. Deployment

- **Melhor k:** O tuning indicou $k = 1$. Isto sugere que as fronteiras de decisão são muito irregulares e específicas.
- **Matriz de Confusão:** O sistema funciona perfeitamente para "Sentado", mas confunde atividades dinâmicas entre si.
- **Comparação:** O Random Forest bateu o k-NN com significância estatística ($p < 0.05$, McNemar), provando ser mais robusto para este dataset de 144 dimensões.

7. Melhorias (Random Forest)

O Random Forest obteve uma Accuracy de $\approx 67\%$. Embora não seja perfeito, mostra melhorias face ao KNN.

Por que funcionou melhor?

- **Seleção de Features Implícita:** O Random Forest consegue ignorar features ruidosas (que atrapalham o cálculo de distância do KNN).
- **Fronteiras não-lineares:** Consegue criar regras de decisão complexas que separam melhor as classes misturadas (4 e 5) do que a simples proximidade geométrica do KNN.

Observação Crítica na Classe 7: Mesmo o Random Forest falhou catastroficamente na classe 7 (F1-Score: 0.14). Isto sugere que o problema da classe 7 não é do modelo, mas sim dos dados (features insuficientes ou má etiquetagem original).

Conclusão

Este trabalho permitiu desenvolver um sistema completo de Reconhecimento de Atividade Humana (HAR), desde o processamento dos dados brutos dos sensores até à criação de um modelo final capaz de classificar novas atividades. O ficheiro submission.csv comprova que o sistema está funcional e pronto a ser utilizado.

Decisões Importantes: A nossa escolha recaiu sobre o modelo Random Forest treinado com a estratégia Between-Subject.

- **Porquê Random Forest?** Os testes estatísticos (McNemar) provaram que este modelo consegue corrigir muitos dos erros que o k-NN cometia, sendo estatisticamente superior.
- **Porquê Between-Subject?** Embora a estratégia Within-Subject desse notas mais altas, percebemos que isso era fruto de "memorizar" os participantes. A estratégia Between-Subject é mais honesta: testa o modelo em pessoas que ele nunca viu, simulando a realidade de uma aplicação comercial.

Análise dos Resultados: Olhando para as previsões finais, tiramos três grandes conclusões:

- **O que funciona bem:** O modelo é excelente a detetar quando a pessoa está parada ou sentada (Atividade 1). As features que extraímos conseguem isolar este estado com grande confiança.
- **Onde há confusão:** As atividades dinâmicas intensas, como correr e subir escadas (Atividades 4 e 5), ainda se confundem bastante. O modelo percebe que há movimento rápido, mas tem dificuldade em distinguir o padrão exato de cada uma.
- **O ponto fraco:** A Atividade 7 é o "calcanhar de Aquiles" do sistema, representando menos de 1

O que faríamos a seguir: O foco principal seria resolver o problema da Atividade 7, recolhendo mais dados ou duplicando artificialmente os existentes (Data Augmentation). Também seria interessante tentar combinar as nossas features manuais com os embeddings da rede neuronal num modelo híbrido, para tentar separar melhor as atividades de corrida e escadas.

Em suma, o sistema cumpre o objetivo: recebe dados de um desconhecido e devolve uma classificação fundamentada, com uma performance sólida nas atividades mais comuns.